UniBot – Sistema Asesor Académico

Documento de Especificación y Planeación · Fase 1

Estudiante: Juan Alejandro Barrero · **Código:** A00418628 · **Docente:** Pablo Cruz
Curso: Algoritmos y programación
Fecha: 26/09/2026

1. Caracterización del sistema

UniBot es una aplicación de consola hecha en Node.js que funciona como un sistema RAG simplificado. Antes de enviarle la pregunta al modelo de IA, el sistema busca en un archivo local la información de la carrera del estudiante y limita las respuestas a los temas permitidos para esa carrera. En la siguiente tabla se describe el sistema desde el enfoque de sistemas:

Componente	Descripción
Objetivo	Dar a los estudiantes de primer ingreso una asesoría académica automática que solo responda sobre los temas de su carrera, y que la universidad pueda controlar quién accede, qué temas se permiten y cuánto cuesta cada sesión.
Entradas	• Código institucional del estudiante y contraseña maestra del administrador (.env). • Preguntas del estudiante en el chat. • Archivos estudiantes.csv (padrón) y carreras.json (contexto: director, edificio, temas permitidos). • Datos de nuevos estudiantes y nuevos temas ingresados por el administrador. • Respuestas de la API del LLM.
Procesos	• Autenticar al estudiante (CSV) y al administrador (.env). • Recuperar el contexto de la carrera y construir el prompt restringido (PromptBuilder). • Consultar el LLM de forma asíncrona y mostrar la respuesta. • Registrar la sesión, acumular caracteres y calcular el costo (caracteres × 2 COP) al cerrar. • Guardar el historial del chat en .txt, administrar estudiantes, temas y sesiones, y controlar los errores.
Salidas	• Respuestas sobre los temas de la carrera, o un mensaje amable cuando la pregunta no está permitida. • Mensajes de acceso concedido o denegado. • estudiantes.csv y carreras.json actualizados. • registro_sesiones.json con el estado y el costo de cada sesión, y el reporte de auditoría. • Historial de cada conversación en logs/AAAA-MM-DD_HH-mm-ss_Codigo_Carrera.txt.
Elementos (subsistemas)	• Interfaz de consola (menú dual). • Acceso y autenticación. • Gestión de datos en archivos (lectores CSV y JSON). • Contexto RAG (PromptBuilder) y comunicación con el LLM (LLMService). • Sesiones y costos, trazabilidad (logs) y control de errores.

Componente	Descripción
Actores y entorno	Los actores son el Estudiante, el Administrador y la API del LLM (sistema externo). El entorno es la universidad con sus planes de estudio, el proveedor del LLM y los archivos locales. Como retroalimentación, la auditoría y los logs le sirven al administrador para ajustar los temas y los accesos.

2. Modelo de proceso y marco de trabajo

Para este proyecto se escogió **Scrum con desarrollo incremental**. La razón principal es que el enunciado ya viene dividido en fases, y en cada una se entrega una versión del programa que funciona y que agrega algo nuevo a la anterior: primero un prototipo en JavaScript, luego la versión en TypeScript con POO y al final la integración con la IA. Además, el profesor revisa cada entrega y puede pedir cambios, y la API del LLM se entrega más adelante, así que tiene sentido dejarla para el último sprint. Con un modelo en cascada habría que hacer todo de una vez y solo se vería el resultado al final, mientras que con Scrum se puede entregar algo útil desde el primer sprint y reorganizar el Product Backlog si algo cambia.

En cuanto a los roles, el profesor cumple el papel de Product Owner, porque define y prioriza los requerimientos, y yo cumplo el de equipo de desarrollo. El avance se va a seguir en un tablero Kanban (Por hacer, En progreso y Hecho), y cada sprint termina con la entrega de su fase:

Sprint	Objetivo	Backlog
Sprint 0 Fase 1	**Pensamiento sistémico e ingeniería de requisitos.** Entender el problema y dejar planeada la solución antes de escribir código.	Caracterización del sistema, justificación del proceso, Product Backlog, diagrama de casos de uso, tablero Kanban y repositorio con README.md.
Sprint 1 Fase 2	**Prototipo estructurado y autenticación.** Tener un prototipo funcional en JavaScript que controle quién entra al sistema.	RF-01, RF-02, RF-03, RF-04.
Sprint 2 Fase 3	**Transición a POO, TypeScript y gestión.** Migrar a TypeScript orientado a objetos y completar el módulo del administrador.	RF-05 a RF-10.
Sprint 3 Fase 4	**Integración de IA, manejo de errores y trazabilidad.** Conectar el chat con el LLM de forma restringida, segura y auditable.	RF-11 a RF-14.

3. Product Backlog

3.1 Requerimientos funcionales

ID	Historia de usuario	Prioridad	Pts	Sprint
RF-01	**Menú principal dual.** Como **usuario de UniBot** quiero un menú principal que me deje elegir entre el rol Estudiante, el rol Administrador o salir, **para** entrar únicamente a las funciones que me corresponden.	Alta	2	1
RF-02	**Autenticación del administrador.** Como **administrador** quiero ingresar con una contraseña maestra almacenada en el archivo .env, **para** que ninguna persona sin autorización modifique accesos, reglas o sesiones.	Alta	3	1

ID	Historia de usuario	Prioridad	Pts	Sprint
RF-03	**Lectura del padrón de estudiantes (CSV).** Como **administrador** quiero que los estudiantes autorizados se lean desde estudiantes.csv, **para** mantener el padrón de acceso sin necesidad de una base de datos.	Alta	3	1
RF-04	**Ingreso del estudiante por código.** Como **estudiante** quiero ingresar con mi código institucional, **para** recibir asesoría con la información de mi propia carrera.	Alta	3	1
RF-05	**Registro de nuevos estudiantes.** Como **administrador** quiero dar de alta a nuevos estudiantes, **para** que los alumnos de primer ingreso puedan usar el asesor desde el primer día.	Alta	3	2
RF-06	**Consulta y gestión de temas permitidos (JSON).** Como **administrador** quiero ver las carreras con sus temas permitidos y agregarles nuevas palabras clave, **para** ampliar lo que el asesor puede responder sin modificar el código.	Alta	5	2
RF-07	**Registro y seguimiento de sesiones.** Como **administrador** quiero que cada ingreso al chat genere un registro de sesión, **para** auditar el uso real del servicio.	Alta	5	2
RF-08	**Cálculo del costo de la sesión.** Como **administrador** quiero conocer el costo de cada sesión, **para** controlar el gasto de la universidad en el servicio de IA.	Alta	3	2
RF-09	**Auditoría de sesiones.** Como **administrador** quiero ver cuántas sesiones están activas y cuántas cerradas, **para** detectar sesiones abandonadas o sin liquidar.	Media	3	2
RF-10	**Cierre forzado de sesiones.** Como **administrador** quiero cerrar de una vez todas las sesiones activas, **para** que ninguna quede sin su costo liquidado.	Media	3	2
RF-11	**Chat con el asesor virtual (LLM).** Como **estudiante** quiero hacer preguntas y recibir respuestas de un asesor con inteligencia artificial, **para** resolver mis dudas sin esperar a un asesor humano.	Alta	8	3
RF-12	**Restricción temática (PromptBuilder).** Como **estudiante** quiero que el asesor me responda solo sobre los temas de mi carrera, **para** no recibir información de otros planes de estudio.	Alta	5	3
RF-13	**Exportación del historial del chat.** Como **administrador** quiero que al terminar el chat se guarde la conversación en un archivo de texto, **para** conservar la trazabilidad de lo que se respondió.	Alta	3	3
RF-14	**Manejo de errores y recuperación.** Como **usuario** quiero que, si falla la API o falta un archivo, el sistema me avise y me devuelva al menú, **para** no tener que reiniciar la aplicación.	Alta	5	3

3.2 Requerimientos no funcionales

ID	Categoría	Requerimiento no funcional
RNF-01	Seguridad	La contraseña maestra y las credenciales de la API se leen desde variables de entorno (.env). No se escriben en el código ni se suben al repositorio.
RNF-02	Robustez	Las lecturas y escrituras de archivos y las llamadas a la API van dentro de bloques try/catch, para que un error no cierre el programa.

ID	Categoría	Requerimiento no funcional
RNF-03	Usabilidad	Los mensajes en consola deben ser claros e indicar si una búsqueda falló, si se negó el acceso o si la operación salió bien. Los errores se muestran en rojo.
RNF-04	Persistencia	No se usa base de datos. La información se guarda en archivos CSV, JSON y TXT dentro de las carpetas data/ y logs/.
RNF-05	Portabilidad	Las rutas se construyen con path.join para funcionar en Windows, macOS y Linux.
RNF-06	Tecnología	Aplicación de consola en Node.js 18 o superior. El prototipo se hace en JavaScript y la versión final en TypeScript con strict: true.
RNF-07	Mantenibilidad	El código se organiza por módulos (modelos, controladores e interfaz), usa POO, nombres en inglés (camelCase y PascalCase) y se documenta con TSDoc.
RNF-08	Desempeño	La comunicación con el LLM es asíncrona (async/await), para que la aplicación no se bloquee mientras espera la respuesta.
RNF-09	Trazabilidad	Cada sesión queda registrada con su costo, y cada conversación se guarda en un archivo .txt con el formato de nombre definido.
RNF-10	Control de versiones	El proyecto se gestiona en un repositorio de Git/GitHub, con al menos 3 commits significativos en las fases 2 y 4, como lo pide el enunciado.

4. Diagrama de casos de uso

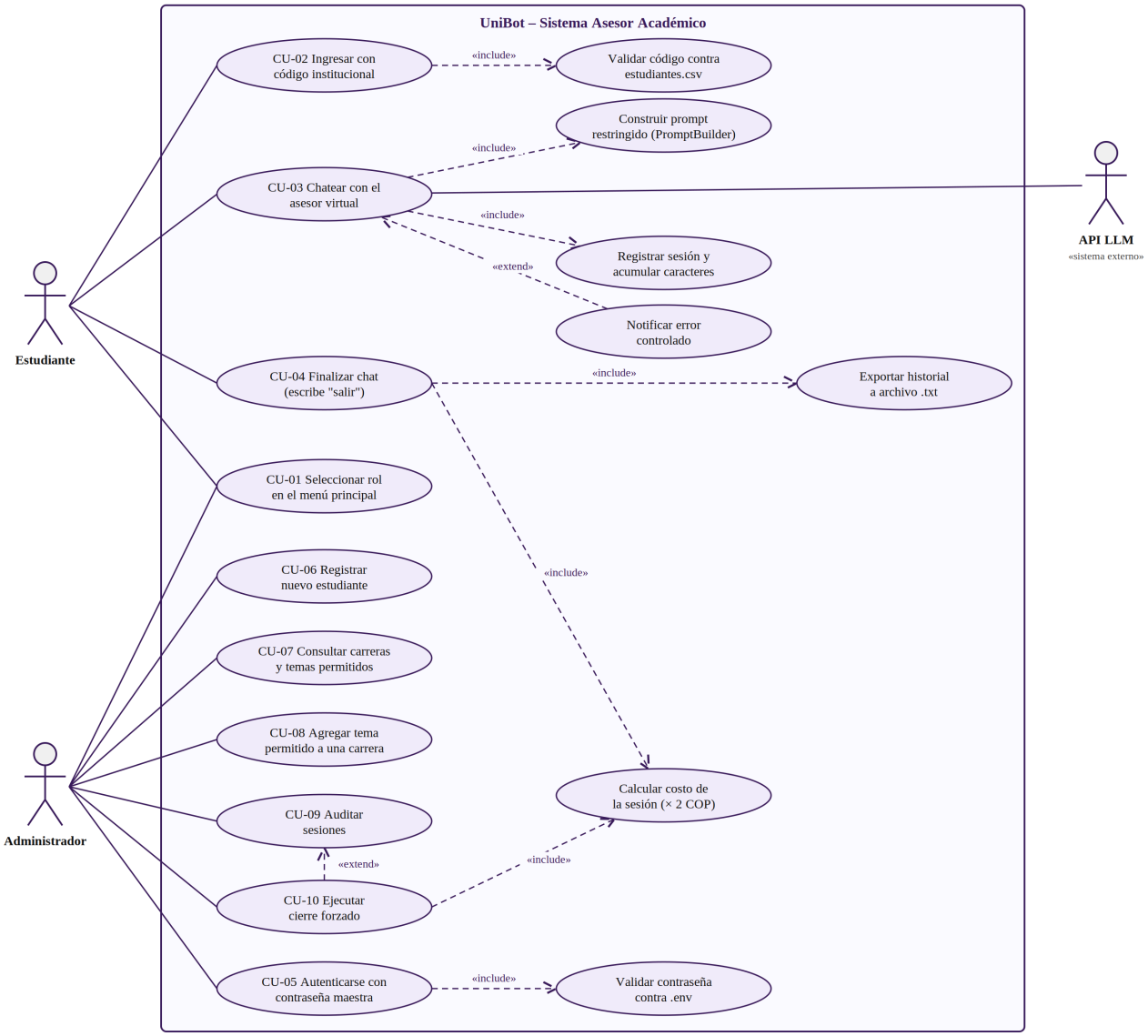


Figura 1. Diagrama de casos de uso de UniBot (UML).